7. WAP to implement a circular queue in c

```
#include <stdio.h>
#include <stdlib.h>

#define MAX_SIZE 5

typedef struct {
   int* elements;
   int front;
   int rear;
   int size;
} CircularQueue;

CircularQueue* createQueue() {
   CircularQueue* queue = (CircularQueue*)malloc(sizeof(CircularQueue));
   queue->elements = (int*)malloc(MAX_SIZE * sizeof(int));
   queue->front = -1;
   queue->rear = -1;
   queue->size = 0;
   return queue;
}

int isFull(CircularQueue* queue) {
   return (queue->front == 0 && queue->rear == MAX_SIZE - 1) || (queue->front ==
queue->rear + 1);
}
```

```
int isEmpty(CircularQueue* queue) {
    return queue->front == -1;
}

void enqueue(CircularQueue* queue, int data) {
    if (isFull(queue)) {
        printf("Queue is full. Cannot enqueue element.\n");
        return;
    }

    if (queue->front == -1)
        queue->front = 0;

    queue->rear = (queue->rear + 1) % MAX_SIZE;
    queue->elements[queue->rear] = data;
    queue->size++;

    printf("Enqueued element: %d\n", data);
}

int dequeue(CircularQueue* queue) {
    if (isEmpty(queue)) {
        printf("Queue is empty. Cannot dequeue element.\n");
        return -1;
    }
```

```
    int data = queue->elements[queue->front];
    if (queue->front == queue->rear)
        queue->front = queue->rear = -1;
    else
        queue->front = (queue->front + 1) % MAX_SIZE;

    queue->size--;

    return data;
}

int main() {
    CircularQueue* queue = createQueue();

    enqueue(queue, 10);
    enqueue(queue, 20);
    enqueue(queue, 30);
    enqueue(queue, 40);
    enqueue(queue, 50);
    enqueue(queue, 60);

    printf("Dequeued element: %d\n", dequeue(queue));
    printf("Dequeued element: %d\n", dequeue(queue));
    printf("Dequeued element: %d\n", dequeue(queue));
    printf("Dequeued element: %d\n", dequeue(queue));
    printf("Dequeued element: %d\n", dequeue(queue));
    printf("Dequeued element: %d\n", dequeue(queue));
```

```
    free(queue->elements);
    free(queue);

    return 0;
}
```

OUTPUT:

```
PS C:\Users\ayanr\Desktop\GU Lab\SEMII\DSA 6 onwards> cd "c:\Users\ayanr\Desktop\GU Lab\SEMII\DSA 6 onwards\" ; if ($?) { gcc Exp_06.c -o Exp_06 } ; if ($?) {
 .\Exp_06 }
Enqueued element: 10
Enqueued element: 20
Enqueued element: 30
Enqueued element: 40
Enqueued element: 50
Queue is full. Cannot enqueue element.
Dequeued element: 10
Dequeued element: 20
Dequeued element: 30
Dequeued element: 40
Dequeued element: 50
Queue is empty. Cannot dequeue element.
Dequeued element: -1
PS C:\Users\ayanr\Desktop\GU Lab\SEMII\DSA 6 onwards>
```

8. Write to get the nth node in linked list

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node
{
    int data;
    struct Node *next;
} Node;

Node *createNode(int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    newNode->data = data;
```

```
    newNode->next = NULL;
    return newNode;
}

Node *insertNode(Node *head, int data)
{
    Node *newNode = createNode(data);
    if (head == NULL)
    {
        head = newNode;
    }
    else
    {
        Node *temp = head;
        while (temp->next != NULL)
        {
            temp = temp->next;
        }
        temp->next = newNode;
    }
    return head;
}

int getNodeValue(Node *head, int position)
{
    if (head == NULL)
    {
```

```
        printf("Error: Linked list is empty.\n");
        return -1;
    }

    Node *temp = head;
    int count = 1;
    while (temp != NULL && count != position)
    {
        temp = temp->next;
        count++;
    }

    if (temp == NULL)
    {
        printf("Error: Position out of range.\n");
        return -1;
    }

    return temp->data;
}

void freeLinkedList(Node *head)
{
    Node *temp;
    while (head != NULL)
    {
        temp = head;
```

```
        head = head->next;
        free(temp);
    }
}

int main()
{
    Node *head = NULL;
    int n, data;

    printf("Enter the number of elements: ");
    scanf("%d", &n);

    printf("Enter the elements:\n");
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &data);
        head = insertNode(head, data);
    }

    int position;
    printf("Enter the position of the node to retrieve: ");
    scanf("%d", &position);

    int nodeValue = getNodeValue(head, position);
    if (nodeValue != -1)
    {
```

```
        printf("Value of the node at position %d: %d\n", position, nodeValue);
    }

    freeLinkedList(head);
    return 0;
}
```

OUTPUT:

```
PS C:\Users\ayanr\Desktop\GU Lab\SEMII\DSA 6 onwards> cd "c:\Users\ayanr\Desktop\GU Lab\SEMII\DSA 6 onwards\" ; if ($?) { gcc Exp_08.c -o Exp_08 } ; if ($?) {
 .\Exp_08 }
Enter the number of elements: 5
Enter the elements:
1
5
7
6
8
Enter the position of the node to retrieve: 3
Value of the node at position 3: 7
PS C:\Users\ayanr\Desktop\GU Lab\SEMII\DSA 6 onwards>
```

9. WAP program to implement binary search tree

```
#include <stdio.h>
#include <stdlib.h>

typedef struct TreeNode
{
    int data;
    struct TreeNode *left;
    struct TreeNode *right;
} TreeNode;

TreeNode *createNode(int data)
{
    TreeNode *newNode = (TreeNode *)malloc(sizeof(TreeNode));
    newNode->data = data;
```

```
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}

TreeNode *insert(TreeNode *root, int data)
{
    if (root == NULL)
    {
        root = createNode(data);
    }
    else if (data <= root->data)
    {
        root->left = insert(root->left, data);
    }
    else
    {
        root->right = insert(root->right, data);
    }
    return root;
}

int search(TreeNode *root, int key)
{
    if (root == NULL)
    {
        return 0;
```

```
    }
    else if (key == root->data)
    {
        return 1;
    }
    else if (key < root->data)
    {
        return search(root->left, key);
    }
    else
    {
        return search(root->right, key);
    }
}

void inorderTraversal(TreeNode *root)
{
    if (root != NULL)
    {
        inorderTraversal(root->left);
        printf("%d ", root->data);
        inorderTraversal(root->right);
    }
}

void freeTree(TreeNode *root)
{
```

```
    if (root != NULL)
    {
        freeTree(root->left);
        freeTree(root->right);
        free(root);
    }
}

int main()
{
    TreeNode *root = NULL;

    root = insert(root, 50);
    root = insert(root, 30);
    root = insert(root, 20);
    root = insert(root, 40);
    root = insert(root, 70);
    root = insert(root, 60);
    root = insert(root, 80);

    printf("Inorder traversal: ");
    inorderTraversal(root);
    printf("\n");

    int key;
    printf("Enter a key to search: ");
    scanf("%d", &key);
```

```
    if (search(root, key))
    {
        printf("Key %d found in the BST.\n", key);
    }
    else
    {
        printf("Key %d not found in the BST.\n", key);
    }

    freeTree(root);

    return 0;
}
```

OUTPUT:

```
PS C:\Users\ayanr\Desktop\GU Lab\SEMII\DSA 6 onwards> cd "c:\Users\ayanr\Desktop\GU Lab\SEMII\DSA 6 onwards\" ; if ($?) { gcc Exp_09.c -o Exp_09 } ; if ($?) {
 .\Exp_09 }
Inorder traversal: 20 30 40 50 60 70 80
Enter a key to search: 50
Key 50 found in the BST.
PS C:\Users\ayanr\Desktop\GU Lab\SEMII\DSA 6 onwards>
```

10. WAP program to implement insertion sort.

```
#include <stdio.h>


void insertionSort(int arr[], int n) {
   int i, j, key;
   for (i = 1; i < n; i++) {
      key = arr[i];
      j = i - 1;


      // Move elements of arr[0..i-1], that are greater than key, to one position ahead
of their current position
      while (j >= 0 && arr[j] > key) {
         arr[j + 1] = arr[j];
         j = j - 1;
      }
      arr[j + 1] = key;
   }
}


void printArray(int arr[], int n) {
   int i;
   for (i = 0; i < n; i++) {
      printf("%d ", arr[i]);
   }
   printf("\n");
}
```

```
int main() {
    int arr[] = {12, 11, 13, 5, 6};
    int n = sizeof(arr) / sizeof(arr[0]);

    printf("Array before sorting: ");
    printArray(arr, n);

    insertionSort(arr, n);

    printf("Array after sorting: ");
    printArray(arr, n);

    return 0;
}
```

OUTPUT:

```
PS C:\Users\ayanr\Desktop\GU Lab\SEMII\DSA 6 onwards> cd "c:\Users\ayanr\Desktop\GU Lab\SEMII\DSA 6 onwards\" ; if ($?) { gcc Exp_10.c -o Exp_10 } ; if ($?) {
 .\Exp_10 }
Array before sorting: 12 11 13 5 6
Array after sorting: 5 6 11 12 13
PS C:\Users\ayanr\Desktop\GU Lab\SEMII\DSA 6 onwards>
```

11. WAP program to implement bubble sort.

```
#include <stdio.h>
void bubbleSort(int arr[], int n)
{
    int i, j, temp;
    for (i = 0; i < n - 1; i++)
```

```
    {
        // Last i elements are already in place
        for (j = 0; j < n - i - 1; j++)
        {
            // Swap adjacent elements if they are in the wrong order
            if (arr[j] > arr[j + 1])
            {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

void printArray(int arr[], int n)
{
    int i;
    for (i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main()
{
```

```
    int arr[] = {64, 34, 25, 12, 22, 11, 90};
    int n = sizeof(arr) / sizeof(arr[0]);

    printf("Array before sorting: ");
    printArray(arr, n);

    bubbleSort(arr, n);

    printf("Array after sorting: ");
    printArray(arr, n);

    return 0;
}
```

OUTPUT:

```
PS C:\Users\ayanr\Desktop\GU Lab\SEMII\DSA 6 onwards> cd "c:\Users\ayanr\Desktop\GU Lab\SEMII\DSA 6 onwards\" ; if ($?) { gcc Exp_11.c -o Exp_11 } ; if ($?) {
 .\Exp_11 }
Array before sorting: 64 34 25 12 22 11 90
Array after sorting: 11 12 22 25 34 64 90
PS C:\Users\ayanr\Desktop\GU Lab\SEMII\DSA 6 onwards>
```

12. WAP program to implement Quick Sort.

```
#include <stdio.h>

void swap(int *a, int *b)
```

```
{
    int temp = *a;
    *a = *b;
    *b = temp;
}

int partition(int arr[], int low, int high)
{
    int pivot = arr[high];
    int i = (low - 1);

    for (int j = low; j <= high - 1; j++)
    {
        if (arr[j] < pivot)
        {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return (i + 1);
}

void quickSort(int arr[], int low, int high)
{
    if (low < high)
    {
```

```
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

void printArray(int arr[], int size)
{
    int i;
    for (i = 0; i < size; i++)
    {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main()
{
    int arr[] = {10, 7, 8, 9, 1, 5};
    int n = sizeof(arr) / sizeof(arr[0]);

    printf("Array before sorting: ");
    printArray(arr, n);

    quickSort(arr, 0, n - 1);

    printf("Array after sorting: ");
```

```
    printArray(arr, n);

    return 0;
}
```

OUTPUT:

```
PS C:\Users\ayanr\Desktop\GU Lab\SEMII\DSA 6 onwards> cd "c:\Users\ayanr\Desktop\GU Lab\SEMII\DSA 6 onwards\" ; if ($?) { gcc Exp_12.c -o Exp_12 } ; if ($?) {
 .\Exp_12 }
Array before sorting: 10 7 8 9 1 5
Array after sorting: 1 5 7 8 9 10
PS C:\Users\ayanr\Desktop\GU Lab\SEMII\DSA 6 onwards>
```

13. WAP program to implement Merge sort.

```
#include <stdio.h>

void merge(int arr[], int left, int middle, int right)
{
    int i, j, k;
    int n1 = middle - left + 1;
    int n2 = right - middle;

    // Create temporary arrays
    int L[n1], R[n2];

    // Copy data to temporary arrays L[] and R[]
    for (i = 0; i < n1; i++)
        L[i] = arr[left + i];
```

```
for (j = 0; j < n2; j++)
   R[j] = arr[middle + 1 + j];

// Merge the temporary arrays back into arr[left..right]
i = 0;    // Initial index of first subarray
j = 0;    // Initial index of second subarray
k = left; // Initial index of merged subarray
while (i < n1 && j < n2)
{
   if (L[i] <= R[j])
   {
      arr[k] = L[i];
      i++;
   }
   else
   {
      arr[k] = R[j];
      j++;
   }
   k++;
}

// Copy the remaining elements of L[], if there are any
while (i < n1)
{
   arr[k] = L[i];
   i++;
```

```
        k++;
    }

    // Copy the remaining elements of R[], if there are any
    while (j < n2)
    {
        arr[k] = R[j];
        j++;
        k++;
    }
}

void mergeSort(int arr[], int left, int right)
{
    if (left < right)
    {
        int middle = left + (right - left) / 2;

        // Sort first and second halves
        mergeSort(arr, left, middle);
        mergeSort(arr, middle + 1, right);

        merge(arr, left, middle, right);
    }
}

void printArray(int arr[], int size)
```

```
{
    int i;
    for (i = 0; i < size; i++)
    {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main()
{
    int arr[] = {12, 11, 13, 5, 6, 7};
    int n = sizeof(arr) / sizeof(arr[0]);

    printf("Array before sorting: ");
    printArray(arr, n);

    mergeSort(arr, 0, n - 1);

    printf("Array after sorting: ");
    printArray(arr, n);

    return 0;
}
```

OUTPUT:

```
PS C:\Users\ayanr\Desktop\GU Lab\SEMII\DSA 6 onwards> cd "c:\Users\ayanr\Desktop\GU Lab\SEMII\DSA 6 onwards\" ; if ($?) { gcc Exp_13.c -o Exp_13 } ; if ($?) {
 .\Exp_13 }
Array before sorting: 12 11 13 5 6 7
Array after sorting: 5 6 7 11 12 13
PS C:\Users\ayanr\Desktop\GU Lab\SEMII\DSA 6 onwards>
```

14. Directed Graph Implementation.

```
#include <stdio.h>

#include <stdlib.h>


#define MAX_VERTICES 100


typedef struct Node

{

   int vertex;

   struct Node *next;

} Node;


typedef struct Graph

{

   Node *adjacencyList[MAX_VERTICES];

   int numVertices;

} Graph;


Node *createNode(int v)

{

   Node *newNode = (Node *)malloc(sizeof(Node));

   newNode->vertex = v;
```

```
    newNode->next = NULL;
    return newNode;
}

Graph *createGraph(int numVertices)
{
    Graph *graph = (Graph *)malloc(sizeof(Graph));
    graph->numVertices = numVertices;

    for (int i = 0; i < numVertices; i++)
    {
        graph->adjacencyList[i] = NULL;
    }

    return graph;
}

void addEdge(Graph *graph, int src, int dest)
{
    Node *newNode = createNode(dest);
    newNode->next = graph->adjacencyList[src];
    graph->adjacencyList[src] = newNode;
}

void printGraph(Graph *graph)
{
    for (int i = 0; i < graph->numVertices; i++)
```

```
    {
        Node *temp = graph->adjacencyList[i];
        printf("Adjacency list of vertex %d: ", i);
        while (temp)
        {
            printf("%d -> ", temp->vertex);
            temp = temp->next;
        }
        printf("NULL\n");
    }
}

int main()
{
    int numVertices = 4;
    Graph *graph = createGraph(numVertices);

    addEdge(graph, 0, 1);
    addEdge(graph, 0, 2);
    addEdge(graph, 1, 2);
    addEdge(graph, 2, 0);
    addEdge(graph, 2, 3);
    addEdge(graph, 3, 3);

    printGraph(graph);

    return 0;
```

}

OUTPUT:

```
PS C:\Users\ayanr\Desktop\GU Lab\SEMII\DSA 6 onwards> cd "c:\Users\ayanr\Desktop\GU Lab\SEMII\DSA 6 onwards\" ; if ($?) { gcc
Exp_14.c -o Exp_14 } ; if ($?) { .\Exp_14 }
Adjacency list of vertex 0: 2 -> 1 -> NULL
Adjacency list of vertex 1: 2 -> NULL
Adjacency list of vertex 2: 3 -> 0 -> NULL
Adjacency list of vertex 3: 3 -> NULL
PS C:\Users\ayanr\Desktop\GU Lab\SEMII\DSA 6 onwards>
```